PASSWORD AUDIT SYSTEM

password_audit.py

```
import abc

class PasswordCheck(abc.ABC):
    """Abstract base class for all password checks."""

    @abc.abstractmethod
    def run_check(self, password):
        """Run validation on a password string.
        Returns:
            tuple: (deduction_points: int, issues: list[str])
        """
        pass


class LengthCheck(PasswordCheck):
    """Checks if password meets minimum length requirement."""

    MIN_LENGTH = 8

    def run_check(self, password):
        if len(password) < self.MIN_LENGTH:
            return (1, [f"too short ({len(password)} chars, need {self.MIN_LENGTH})"])
        return (0, [])


class ComplexityCheck(PasswordCheck):
    """Checks for presence of uppercase, lowercase, digit, special char."""

    SPECIAL_CHARS = "!@#$%^&*()-_=+[]{}|;:,.<>?/~`"

    def run_check(self, password):
        issues = []
        deduction = 0

        if not any(c.isupper() for c in password):
            issues.append("missing uppercase letter")
            deduction += 1
        if not any(c.islower() for c in password):
            issues.append("missing lowercase letter")
            deduction += 1
        if not any(c.isdigit() for c in password):
            issues.append("missing digit")
            deduction += 1
        if not any(c in self.SPECIAL_CHARS for c in password):
            issues.append("missing special character")
            deduction += 1

        return (deduction, issues)


class PatternCheck(PasswordCheck):
    """Checks against common weak passwords (frozenset blacklist)."""

    WEAK_PATTERNS = frozenset({
        "password", "password123", "123456", "12345678",
        "qwerty", "abc123", "monkey", "letmein", "admin",
        "welcome", "football", "iloveyou", "sunshine",
        "princess", "passw0rd", "111111", "000000", "666666",
        "dragon", "master", "shadow", "trustno1"
    })

    def run_check(self, password):
        issues = []
        deduction = 0

        if password.lower() in self.WEAK_PATTERNS:
            issues.append("matches a known weak password")
            deduction = 4
        elif password.isdigit():
            issues.append("contains only digits")
            deduction = max(deduction, 3)
        elif len(password) < 4:
            issues.append("too short (less than 4 characters)")
            deduction = max(deduction, 2)

        return (deduction, issues)
```

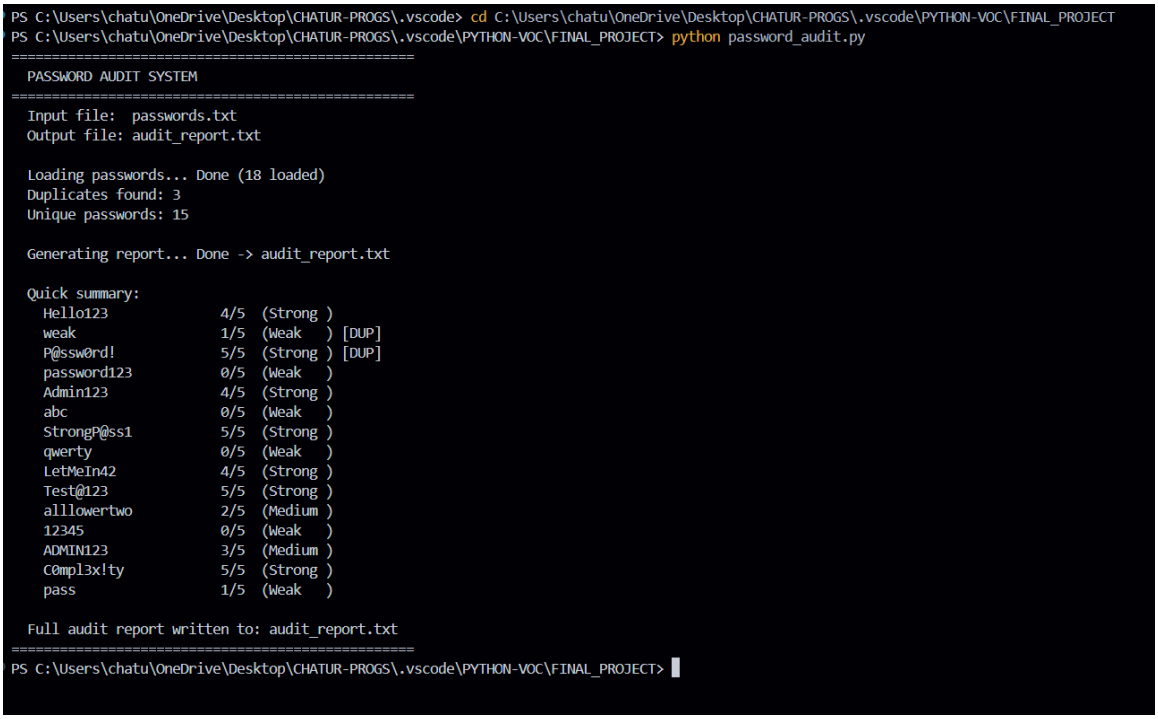
```
class Password:
    """Stores a single password and its audit results.
    Encapsulation: __score and __issues are private (name-mangled), exposed via @property.
    """

    MAX_SCORE = 5

    def __init__(self, password_str):
        self.__password = password_str
        self.__score = self.MAX_SCORE
        self.__issues = []
        self.__is_duplicate = False

    @property
    def password(self):
        return self.__password

    @property
    def score(self):
        return self.__score

    @property
    def issues(self):
        return list(self.__issues)

    @property
    def is_duplicate(self):
        return self.__is_duplicate

    @is_duplicate.setter
    def is_duplicate(self, value):
        self.__is_duplicate = bool(value)

    def analyze(self):
        """Run all password checks (polymorphism in action)."""
        self.__score = self.MAX_SCORE
        self.__issues = []

        checks = [LengthCheck(), ComplexityCheck(), PatternCheck()]
        for check in checks:
            deduction, check_issues = check.run_check(self.__password)
            self.__score -= deduction
            self.__issues.extend(check_issues)

        self.__score = max(0, self.__score)
        return self

    @property
    def strength(self):
        if self.__score >= 4:
            return "Strong"
        elif self.__score >= 2:
            return "Medium"
        else:
            return "Weak"

    def __str__(self):
        dup = " [DUPLICATE]" if self.__is_duplicate else ""
        return f"{self.__password} | Score: {self.__score}/5 ({self.strength}){dup}"

    def __repr__(self):
        return f"Password('{self.__password}', score={self.__score})"


class PasswordAuditor:
    """Orchestrates loading, checking, and analyzing bulk passwords.
    Uses:
        - List: stores all passwords from file
        - Set: detects duplicates
        - Dict: maps password string -> Password object
        - Comprehensions: filters by strength category
    """

    def __init__(self, filepath):
        self._filepath = filepath
        self._passwords = []
        self._password_objects = []
        self._results_dict = {}
        self._duplicate_set = set()

    def load_passwords(self):
        try:
            with open(self._filepath, 'r') as f:
                lines = f.readlines()
        except FileNotFoundError:
            raise FileNotFoundError(f"ERROR: File '{self._filepath}' not found.")

        if not lines:
            raise ValueError("ERROR: Password file is empty.")
```

```
        self._passwords = [line.strip() for line in lines if line.strip()]

        if not self._passwords:
            raise ValueError("ERROR: No valid passwords found in file.")

        seen = set()
        for pwd in self._passwords:
            if pwd in seen:
                self._duplicate_set.add(pwd)
            seen.add(pwd)

        for pwd in self._passwords:
            pw_obj = Password(pwd)
            pw_obj.analyze()
            pw_obj.is_duplicate = pwd in self._duplicate_set
            self._password_objects.append(pw_obj)
            self._results_dict[pwd] = pw_obj

        return self

    @property
    def password_count(self):
        return len(self._passwords)

    @property
    def unique_count(self):
        return len(set(self._passwords))

    @property
    def duplicate_count(self):
        return self.password_count - self.unique_count

    @property
    def duplicate_passwords(self):
        return sorted(self._duplicate_set)

    @property
    def password_objects(self):
        return list(self._password_objects)

    def get_results_as_tuples(self):
        results = []
        for pwd, pw_obj in self._results_dict.items():
            results.append((
                pwd,
                pw_obj.score,
                list(pw_obj.issues),
                pw_obj.strength,
                pw_obj.is_duplicate
            ))
        return results

    def get_result(self, password):
        return self._results_dict.get(password, None)

    def get_passwords_by_strength(self, category):
        return [pw for pw in self._password_objects if pw.strength == category]


class AuditReport:
    """Generates a formatted audit report file."""

    def __init__(self, auditor, output_path="audit_report.txt"):
        self._auditor = auditor
        self._output_path = output_path

    def generate(self):
        try:
            with open(self._output_path, 'w') as f:
                f.write(self._build_content())
            return self._output_path
        except IOError as e:
            raise IOError(f"ERROR: Failed to write report - {e}")

    def _build_content(self):
        a = self._auditor
        lines = []
        lines.append("=" * 60)
        lines.append("                    PASSWORD AUDIT REPORT")
        lines.append("=" * 60)
        lines.append(f"  Total passwords loaded:    {a.password_count}")
        lines.append(f"  Unique passwords:          {a.unique_count}")
        lines.append(f"  Duplicate passwords:       {a.duplicate_count}")
        lines.append("=" * 60)

        if a.duplicate_passwords:
            lines.append("\n  --- DUPLICATE PASSWORDS ---")
            for pwd in a.duplicate_passwords:
```

```
                lines.append(f"    * {pwd}")

        results = a.get_results_as_tuples()
        lines.append("\n" + "-" * 60)
        lines.append("  PASSWORD AUDIT DETAILS")
        lines.append("-" * 60)

        for pwd, score, issues, strength, is_dup in results:
            dup_tag = " [DUPLICATE]" if is_dup else ""
            lines.append(f"\n  Password: {pwd}{dup_tag}")
            lines.append(f"  Score:    {score}/5 ({strength})")
            if issues:
                for issue in issues:
                    lines.append(f"    - {issue}")
            else:
                lines.append(f"    (no issues)")

        weak_list = [t for t in results if t[3] == "Weak"]
        medium_list = [t for t in results if t[3] == "Medium"]
        strong_list = [t for t in results if t[3] == "Strong"]

        lines.append("\n" + "-" * 60)
        lines.append("  SUMMARY BY STRENGTH")
        lines.append("-" * 60)
        lines.append(f"  Weak passwords:   {len(weak_list)}")
        for t in weak_list:
            lines.append(f"    - {t[0]} (Score: {t[1]}/5)")
        lines.append(f"  Medium passwords: {len(medium_list)}")
        for t in medium_list:
            lines.append(f"    - {t[0]} (Score: {t[1]}/5)")
        lines.append(f"  Strong passwords: {len(strong_list)}")
        for t in strong_list:
            lines.append(f"    - {t[0]} (Score: {t[1]}/5)")

        lines.append("\n" + "-" * 60)
        lines.append("  ALL RESULTS (Tuples)")
        lines.append("-" * 60)
        for t in results:
            lines.append(f"    {t}")

        lines.append("\n" + "=" * 60)
        lines.append("                        END OF REPORT")
        lines.append("=" * 60)

        return "\n".join(lines)


def main():
    import sys

    input_file = sys.argv[1] if len(sys.argv) > 1 else "passwords.txt"
    output_file = "audit_report.txt"

    print("=" * 50)
    print("  PASSWORD AUDIT SYSTEM")
    print("=" * 50)
    print(f"  Input file:  {input_file}")
    print(f"  Output file: {output_file}")
    print()

    try:
        print("  Loading passwords...", end=" ")
        auditor = PasswordAuditor(input_file)
        auditor.load_passwords()
        print(f"Done ({auditor.password_count} loaded)")

        print(f"  Duplicates found: {auditor.duplicate_count}")
        print(f"  Unique passwords: {auditor.unique_count}")
        print()

        print("  Generating report...", end=" ")
        report = AuditReport(auditor, output_file)
        path = report.generate()
        print(f"Done -> {path}")

        results = auditor.get_results_as_tuples()
        print()
        print("  Quick summary:")
        for pwd, score, issues, strength, is_dup in results:
            dup = " [DUP]" if is_dup else ""
            print(f"    {pwd:20s}  {score}/5  ({strength:7s}){dup}")

        print()
        print(f"  Full audit report written to: {path}")
        print("=" * 50)

    except (FileNotFoundError, ValueError, IOError) as e:
        print(f"\n  ERROR: {e}")
        print(f"  Create a file named '{input_file}' with one password per line.")
```

```
if __name__ == "__main__":
    main()
```

output-



test_password_audit.py

```
import os
import sys
import tempfile
import unittest

from password_audit import (
    Password,
    PasswordCheck,
    LengthCheck,
    ComplexityCheck,
    PatternCheck,
    PasswordAuditor,
    AuditReport,
)


class TestLengthCheck(unittest.TestCase):
    def setUp(self):
        self.check = LengthCheck()

    def test_long_password_passes(self):
        deduction, issues = self.check.run_check("LongEnough1")
        self.assertEqual(deduction, 0)
        self.assertEqual(issues, [])

    def test_short_password_fails(self):
        deduction, issues = self.check.run_check("Ab1!")
        self.assertEqual(deduction, 1)
        self.assertIn("too short", issues[0])


class TestComplexityCheck(unittest.TestCase):
    def setUp(self):
        self.check = ComplexityCheck()

    def test_all_complexity_present(self):
        pwd = "Ab1!"
        deduction, issues = self.check.run_check(pwd)
        self.assertEqual(deduction, 0)
        self.assertEqual(issues, [])

    def test_missing_uppercase(self):
        deduction, issues = self.check.run_check("abc1!")
        self.assertEqual(deduction, 1)
        self.assertIn("missing uppercase letter", issues)

    def test_missing_lowercase(self):
        deduction, issues = self.check.run_check("ABC1!")
        self.assertEqual(deduction, 1)
        self.assertIn("missing lowercase letter", issues)

    def test_missing_digit(self):
        deduction, issues = self.check.run_check("Abc!")
        self.assertEqual(deduction, 1)
        self.assertIn("missing digit", issues)

    def test_missing_special(self):
        deduction, issues = self.check.run_check("Abc1")
        self.assertEqual(deduction, 1)
        self.assertIn("missing special character", issues)

    def test_all_missing(self):
        deduction, issues = self.check.run_check("   ")
        self.assertEqual(deduction, 4)


class TestPatternCheck(unittest.TestCase):
    def setUp(self):
        self.check = PatternCheck()

    def test_weak_pattern_detected(self):
        deduction, issues = self.check.run_check("password")
        self.assertEqual(deduction, 4)
        self.assertIn("matches a known weak password", issues)

    def test_weak_pattern_case_insensitive(self):
        deduction, issues = self.check.run_check("PASSWORD")
        self.assertEqual(deduction, 4)

    def test_all_digits(self):
        deduction, issues = self.check.run_check("12345")
        self.assertTrue(deduction >= 3)
        self.assertIn("contains only digits", issues)

    def test_strong_password_no_issues(self):
        deduction, issues = self.check.run_check("Tr0ub4dor&3")
```

```
        self.assertEqual(deduction, 0)
        self.assertEqual(issues, [])

    def test_frozenset_is_immutable(self):
        with self.assertRaises(AttributeError):
            PatternCheck.WEAK_PATTERNS.add("new_weak_pwd")


class TestPassword(unittest.TestCase):
    def test_encapsulation(self):
        pw = Password("Test@123")
        with self.assertRaises(AttributeError):
            _ = pw._score
        with self.assertRaises(AttributeError):
            _ = pw._issues

    def test_property_getters(self):
        pw = Password("Test@123")
        self.assertEqual(pw.password, "Test@123")
        pw.analyze()
        self.assertIsInstance(pw.score, int)
        self.assertIsInstance(pw.issues, list)

    def test_is_duplicate_setter(self):
        pw = Password("Test@123")
        pw.is_duplicate = True
        self.assertTrue(pw.is_duplicate)
        pw.is_duplicate = False
        self.assertFalse(pw.is_duplicate)

    def test_analyze_returns_self(self):
        pw = Password("Test@123")
        self.assertIs(pw.analyze(), pw)

    def test_strong_password(self):
        pw = Password("Str0ng!p@$s").analyze()
        self.assertEqual(pw.strength, "Strong")
        self.assertGreaterEqual(pw.score, 4)

    def test_medium_password(self):
        pw = Password("Hello123").analyze()
        self.assertIn(pw.strength, ("Medium", "Strong"))

    def test_weak_password(self):
        pw = Password("abc").analyze()
        self.assertEqual(pw.strength, "Weak")
        self.assertLess(pw.score, 2)

    def test_score_never_negative(self):
        pw = Password("password").analyze()
        self.assertGreaterEqual(pw.score, 0)


class TestPasswordAuditor(unittest.TestCase):
    def setUp(self):
        self.temp_dir = tempfile.mkdtemp()
        self.test_file = os.path.join(self.temp_dir, "test_passwords.txt")

    def _create_file(self, lines):
        with open(self.test_file, 'w') as f:
            f.write("\n".join(lines) + "\n")

    def test_load_passwords_list(self):
        self._create_file(["Hello123", "Test@1", "abc"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        self.assertIsInstance(auditor._passwords, list)
        self.assertEqual(len(auditor._passwords), 3)

    def test_duplicate_detection_set(self):
        self._create_file(["Hello123", "abc", "Hello123"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        self.assertEqual(auditor.duplicate_count, 1)
        self.assertIn("Hello123", auditor.duplicate_passwords)

    def test_results_dictionary(self):
        self._create_file(["Hello123", "Test@1"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        self.assertIsInstance(auditor._results_dict, dict)
        self.assertIn("Hello123", auditor._results_dict)

    def test_get_results_as_tuples(self):
        self._create_file(["Hello123"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        tuples = auditor.get_results_as_tuples()
        self.assertIsInstance(tuples, list)
        pwd, score, issues, strength, is_dup = tuples[0]
        self.assertIsInstance(pwd, str)
        self.assertIsInstance(score, int)
        self.assertIsInstance(issues, list)
        self.assertIsInstance(strength, str)
```

```
        self.assertIsInstance(is_dup, bool)

    def test_comprehension_filter(self):
        self._create_file(["abc", "Hello123", "Str0ng!p@$s"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        weak = auditor.get_passwords_by_strength("Weak")
        self.assertIsInstance(weak, list)
        for pw in weak:
            self.assertEqual(pw.strength, "Weak")

    def test_file_not_found(self):
        auditor = PasswordAuditor("nonexistent.txt")
        with self.assertRaises(FileNotFoundError):
            auditor.load_passwords()

    def test_empty_file(self):
        self._create_file([])
        auditor = PasswordAuditor(self.test_file)
        with self.assertRaises(ValueError):
            auditor.load_passwords()

    def test_no_valid_passwords(self):
        self._create_file(["", "  ", "\t"])
        auditor = PasswordAuditor(self.test_file)
        with self.assertRaises(ValueError):
            auditor.load_passwords()

    def test_properties(self):
        self._create_file(["abc", "abc", "123"])
        auditor = PasswordAuditor(self.test_file).load_passwords()
        self.assertEqual(auditor.password_count, 3)
        self.assertEqual(auditor.unique_count, 2)
        self.assertEqual(auditor.duplicate_count, 1)


class TestAuditReport(unittest.TestCase):
    def setUp(self):
        self.temp_dir = tempfile.mkdtemp()
        self.pwd_file = os.path.join(self.temp_dir, "passwords.txt")
        self.report_file = os.path.join(self.temp_dir, "report.txt")
        with open(self.pwd_file, 'w') as f:
            f.write("Hello123\nabc\n")

    def test_generate_report(self):
        auditor = PasswordAuditor(self.pwd_file).load_passwords()
        report = AuditReport(auditor, self.report_file)
        path = report.generate()
        self.assertTrue(os.path.exists(path))
        with open(path, 'r') as f:
            content = f.read()
        self.assertIn("PASSWORD AUDIT REPORT", content)

    def test_report_contains_expected_sections(self):
        auditor = PasswordAuditor(self.pwd_file).load_passwords()
        report = AuditReport(auditor, self.report_file)
        path = report.generate()
        with open(path, 'r') as f:
            content = f.read()
        self.assertIn("PASSWORD AUDIT DETAILS", content)
        self.assertIn("SUMMARY BY STRENGTH", content)
        self.assertIn("ALL RESULTS (Tuples)", content)
        self.assertIn("END OF REPORT", content)

    def test_report_write_error(self):
        auditor = PasswordAuditor(self.pwd_file).load_passwords()
        report = AuditReport(auditor, "/invalid/path/report.txt")
        with self.assertRaises(IOError):
            report.generate()


class TestSyllabusTopics(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.temp_dir = tempfile.mkdtemp()
        cls.pwd_file = os.path.join(cls.temp_dir, "passwords.txt")
        with open(cls.pwd_file, 'w') as f:
            f.write("Hello123\n")

    def test_inheritance_hierarchy(self):
        self.assertTrue(issubclass(LengthCheck, PasswordCheck))
        self.assertTrue(issubclass(ComplexityCheck, PasswordCheck))
        self.assertTrue(issubclass(PatternCheck, PasswordCheck))

    def test_abstraction_enforced(self):
        with self.assertRaises(TypeError):
            PasswordCheck()

    def test_polymorphism(self):
        checks = [LengthCheck(), ComplexityCheck(), PatternCheck()]
        for check in checks:
```

```
            result = check.run_check("Test@123")
            self.assertIsInstance(result, tuple)
            self.assertEqual(len(result), 2)

    def test_encapsulation_private_members(self):
        pw = Password("Test@123")
        self.assertFalse(hasattr(pw, '_score'))

    def test_variable_types(self):
        pw = Password("Test@123").analyze()
        self.assertIsInstance(pw.password, str)
        self.assertIsInstance(pw.score, int)
        self.assertIsInstance(pw.is_duplicate, bool)

    def test_conditional_present_in_strength(self):
        pw = Password("Test@123").analyze()
        self.assertIn(pw.strength, ("Strong", "Medium", "Weak"))

    def test_file_handling(self):
        self.assertTrue(os.path.exists(self.pwd_file))
        with open(self.pwd_file, 'r') as f:
            content = f.read()
        self.assertIsInstance(content, str)

    def test_frozenset_immutable(self):
        fs = PatternCheck.WEAK_PATTERNS
        self.assertIsInstance(fs, frozenset)


if __name__ == "__main__":
    unittest.main()
```

output-

```
Show Command Actions

Command executed 30 secs ago and took 697ms

ATUR-PROGS\.vscode\PYTHON-VOC\FINAL_PROJECT> python password_audit.py
==========
PS C:\Users\chatu\OneDrive\Desktop\CHATUR-PROGS\.vscode\PYTHON-VOC\FINAL_PROJECT> python -m unittest test_password_audit.py -v
test_generate_report (test_password_audit.TestAuditReport.test_generate_report) ... ok
test_report_contains_expected_sections (test_password_audit.TestAuditReport.test_report_contains_expected_sections) ... ok
test_report_write_error (test_password_audit.TestAuditReport.test_report_write_error) ... ok
test_all_complexity_present (test_password_audit.TestComplexityCheck.test_all_complexity_present) ... ok
test_all_missing (test_password_audit.TestComplexityCheck.test_all_missing) ... ok
test_missing_digit (test_password_audit.TestComplexityCheck.test_missing_digit) ... ok
test_missing_lowercase (test_password_audit.TestComplexityCheck.test_missing_lowercase) ... ok
test_missing_special (test_password_audit.TestComplexityCheck.test_missing_special) ... ok
test_missing_uppercase (test_password_audit.TestComplexityCheck.test_missing_uppercase) ... ok
test_long_password_passes (test_password_audit.TestLengthCheck.test_long_password_passes) ... ok
test_short_password_fails (test_password_audit.TestLengthCheck.test_short_password_fails) ... ok
test_analyze_returns_self (test_password_audit.TestPassword.test_analyze_returns_self) ... ok
test_encapsulation (test_password_audit.TestPassword.test_encapsulation) ... ok
test_is_duplicate_setter (test_password_audit.TestPassword.test_is_duplicate_setter) ... ok
test_medium_password (test_password_audit.TestPassword.test_medium_password) ... ok
test_property_getters (test_password_audit.TestPassword.test_property_getters) ... ok
test_score_never_negative (test_password_audit.TestPassword.test_score_never_negative) ... ok
test_strong_password (test_password_audit.TestPassword.test_strong_password) ... ok
test_weak_password (test_password_audit.TestPassword.test_weak_password) ... ok
test_comprehension_filter (test_password_audit.TestPasswordAuditor.test_comprehension_filter) ... ok
test_duplicate_detection_set (test_password_audit.TestPasswordAuditor.test_duplicate_detection_set) ... ok
test_empty_file (test_password_audit.TestPasswordAuditor.test_empty_file) ... ok
test_file_not_found (test_password_audit.TestPasswordAuditor.test_file_not_found) ... ok
test_get_results_as_tuples (test_password_audit.TestPasswordAuditor.test_get_results_as_tuples) ... ok
test_load_passwords_list (test_password_audit.TestPasswordAuditor.test_load_passwords_list) ... ok
test_no_valid_passwords (test_password_audit.TestPasswordAuditor.test_no_valid_passwords) ... ok
test_properties (test_password_audit.TestPasswordAuditor.test_properties) ... ok
test_results_dictionary (test_password_audit.TestPasswordAuditor.test_results_dictionary) ... ok
test_all_digits (test_password_audit.TestPatternCheck.test_all_digits) ... ok
test_frozenset_is_immutable (test_password_audit.TestPatternCheck.test_frozenset_is_immutable) ... ok
test_strong_password_no_issues (test_password_audit.TestPatternCheck.test_strong_password_no_issues) ... ok
test_weak_pattern_case_insensitive (test_password_audit.TestPatternCheck.test_weak_pattern_case_insensitive) ... ok
test_weak_pattern_detected (test_password_audit.TestPatternCheck.test_weak_pattern_detected) ... ok
test_abstraction_enforced (test_password_audit.TestSyllabusTopics.test_abstraction_enforced) ... ok
test_conditional_present_in_strength (test_password_audit.TestSyllabusTopics.test_conditional_present_in_strength) ... ok
test_encapsulation_private_members (test_password_audit.TestSyllabusTopics.test_encapsulation_private_members) ... ok
test_file_handling (test_password_audit.TestSyllabusTopics.test_file_handling) ... ok
test_frozenset_immutable (test_password_audit.TestSyllabusTopics.test_frozenset_immutable) ... ok
test_inheritance_hierarchy (test_password_audit.TestSyllabusTopics.test_inheritance_hierarchy) ... ok
test_polymorphism (test_password_audit.TestSyllabusTopics.test_polymorphism) ... ok
test_variable_types (test_password_audit.TestSyllabusTopics.test_variable_types) ... ok

----------------------------------------------------------------------
Ran 41 tests in 0.386s

OK
PS C:\Users\chatu\OneDrive\Desktop\CHATUR-PROGS\.vscode\PYTHON-VOC\FINAL_PROJECT>
```

passwords.txt

```
Hello123
weak
P@ssw0rd!
password123
Admin123
P@ssw0rd!
abc
StrongP@ss1
qwerty
LetMeIn42
weak
Test@123
alllowertwo
P@ssw0rd!
12345
ADMIN123
C0mpl3x!ty
pass
```